

Week 7 — Lecture Notes (Instructor)

Form Submission & Asynchronous Operations

Phase 3: Interface Binding · **AI Zone: ON** — **required tool, prompt logs mandatory** **Session length:** ~90 min lecture + handoff to lab **Stack:** Concepts and pseudocode primary; a Blade + Fetch/Axios example included.

Learning Objectives

By the end of this session, students can:

1. Explain how a form collects and submits user input.
2. Distinguish synchronous (page-reload) from asynchronous (Fetch/Axios) submission.
3. Trace the client-side request/response cycle.
4. Handle the async lifecycle — loading, success, error — in the UI.
5. Bind a Create/Update form to a backend controller endpoint.

Session Outline

Time	Segment
0:00-0:06	Recap: views render — now make them send
0:06-0:22	Part A — Anatomy of a form
0:22-0:38	Part B — Synchronous vs. asynchronous submission
0:38-0:41	Discussion break 1
0:41-0:57	Part C — The client-side round trip
0:57-1:12	Part D — The async lifecycle & UI states
1:12-1:14	Discussion break 2
1:14-1:26	Part E — Binding to your controllers
1:26-1:30	Handoff to lab

Recap & Framing (6 min)

Last week your views learned to *render*. This week they learn to *talk back* — to send user input to the logic layer you built in Phase 2. This is the moment the two halves of the app connect: the interface finally drives the controllers. AI stays on; keep logging prompts.

Part A — Anatomy of a Form (16 min)

A form is how the UI collects input. Its parts:

- **Fields & labels** — the inputs, each clearly labeled.
- **The submit control** — the button that sends it.
- **Method & action** (traditional) — where and how the data goes.

Client-side validation is UX, not safety. Recall Week 4: the server is the real gate. Client-side checks (required fields, number ranges) give instant feedback and save a round trip, but they can be bypassed — so the server still validates. Frame client validation as a courtesy to the user, not a security measure.

Part B — Synchronous vs. Asynchronous Submission (16 min)

Two ways a form sends data:

- **Synchronous (traditional POST)** — the browser submits the form, the whole page reloads with the server's response. Simple, but the page flashes and state is lost.
- **Asynchronous (Fetch / Axios)** — JavaScript sends the request in the background; the page does **not** reload. You get the response and update just the part of the UI that changed.

Why async matters for modern UX: no jarring reload, the user keeps their place, and you can show loading and error states inline. Most modern apps submit asynchronously.

Tie to Week 3's monolithic vs. decoupled: a decoupled app almost always submits async to its API; a monolith may use traditional POSTs. Either is valid — teach both, let teams match their pattern.

Part C — The Client-Side Round Trip (16 min)

This is Week 3's request/response lifecycle — seen from the **client** this time.

```
[ form ] --collect--> [ Fetch/Axios ] --request--> [ your controller ]
[ UI   ] <--update--- [ handle result ] <--response-- [ your controller ]
```

Pseudocode:

```
on submit:
  data = collect form fields
  response = await send(POST, "/orders", data) # Fetch or Axios
  if response.ok: update the UI with the new record
  else:          show the errors from the response
```

await is the key new idea. The request takes time; the code waits for the response without freezing the page. Everything after **await** runs when the server answers.

Part D — The Async Lifecycle & UI States (15 min)

Every async request moves through three states — and each maps to a UI you already wireframed:

- **Pending (loading)** — request sent, waiting. Disable the submit button, show a spinner. (This is the loading state from Week 2.)
- **Fulfilled (success)** — response OK (201/200). Update the UI, clear the form, maybe show a confirmation.
- **Rejected (error)** — response not OK (422 validation, 500 server). Show the error; for a 422, show the field-level messages your controllers return.

```
button.disable(); showSpinner()           # pending
response = await send(...)
if response.status == 201: addToList(response.data); resetForm() # fulfilled
else if response.status == 422: showFieldErrors(response.error) # rejected
button.enable(); hideSpinner()
```

This is why Week 4's standardized error shape matters: the client can display a 422 uniformly because every controller answers the same way.

Discussion break 2: "Why disable the submit button while the request is pending?" (Prevents double-submits — creating the same record twice.)

Part E — Binding to Your Controllers (12 min)

Now connect the halves. The form's async request targets the exact endpoints you built in Phase 2:

- **Create form** → `POST /orders` → your `createOrder` controller.
- **Update form** → `PUT /orders/:id` → your `updateOrder` controller, with the form pre-filled from the existing record.

The form sends data (usually JSON), your controller validates and processes it, and returns the standardized response — which the form handles per the lifecycle above. **The app is now end-to-end:** click in the UI, data reaches the database, the UI updates.

In Laravel (one concrete example)

An async submit with Fetch, posting to the controller from Phase 2:

```
<form id="order-form">
  <input name="item"> <input name="qty" type="number">
  <button type="submit">Save</button>
</form>
```

```
document.querySelector('#order-form').addEventListener('submit', async (e) => {
```

```
e.preventDefault(); // stop the full-page reload
const data = { item: item.value, qty: qty.value };
const res = await fetch('/orders', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(data),
});
if (res.status === 201) { /* update UI */ }
else if (res.status === 422) { /* show field errors */ }
});
```

The `422` branch consumes the exact error shape your Form Request returns. The concept — collect, send async, handle the response — transfers to Axios, React, Vue, Angular.

Handoff to Lab (4 min)

Teams bind their Create and Update forms to the Phase 2 controllers: send data async, handle loading/success/error, and confirm records actually persist. AI is on — use it to scaffold the fetch logic, review it, and log prompts. Point them to the Week 7 Lab Handout.

Instructor Notes / Common Pitfalls

- **Double-submit bugs.** If teams don't disable the button while pending, they'll create duplicate records. Make it a requirement.
- **Swallowed errors.** Teams handle the success path and ignore the 422 branch — so failures look like nothing happened. Require visible error handling.
- **Client validation mistaken for safety.** Reinforce that the server still validates (Week 4). Client-side is UX only.
- **CSRF / headers (framework-specific).** If a team's stack needs a CSRF token or specific headers, that's a real-world gotcha — let them hit it and resolve it; it's good learning.
- **Prompt logs continue.** Fetch/async logic is a common thing to scaffold with AI — make sure it's reviewed and logged.